SISTEMAS COMPUTACIONAIS AVANÇADOS (SISTCA)

ADVANCED COMPUTING SYSTEMS

LICENCIATURA EM TELECOMUNICAÇÕES E INFORMÁTICA

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO
POLITÉCNICO DO PORTO

Lab Classes Script:

Running LLMs Locally with Ollama and Open WebUI

Version Control

Version number	Date issued	Authors	Update information
V1.0	<March> 2025	André Borges (1211254) Beatriz Lago (1211006) Francisco Silva (1211084) Raul Reis (1211524) Supervision: Prof. <Paula Viana>	Original version of the script.

Tabela de Conteúdos

1. Introdução 1

1.1 Contexto Científico-Tecnológico 1

1.2 Motivação 1

1.3 Objetivos 2

1.4 Estrutura 2

2. Conceitos Teóricos 3

2.1 LLMs (Large Language Models) 3

2.2 WSL (Windows Subsystem for Linux) 3

2.3 Docker 3

2.4 Estado-de-Arte 3

2.4.1 Ollama 3

2.4.2 LM Studio 4

2.4.3 vLLM 5

2.4.4 Koboldcpp 6

2.4.5 Comparação das Ferramentas 6

2.4.6 Open WebUI 7

3. Perspetivas da tecnologia 7

3.1 Instalação do Ollama e Open WebUI 8

3.1.1 Instalação WSL (Apenas para Utilizadores Windows) 8

3.1.2 Configuração e Instalação Ollama 9

3.1.3 Configuração e Instalação *Open WebUI* 11

3.2 Primeiros passos 15

3.2.1 Conceitos essenciais para a instalação de LLMs 15

3.2.2 Instalação Primeiro LLM - Método 1 16

3.2.3 Instalação Primeiro *LLM* - Método 2 18

4. Tutorial 19

4.1 Criar um LLM Personalizado - "SISTCA-Team4-Nutrition" 19
4.2 Publicar o LLM Criado no Website do Ollama 23
5. Exercícios 26
5.1 Problema A 26
5.1.1 Resolução A 26
5.2 Problema B 28
5.2.1 Resolução B 28
6. Desafio 29
7. Referências 31
8. Anexos 31

1. Introdução

1.1 Contexto Científico-Tecnológico

Neste relatório, o tema abordado será *LLMs (Large Language Models)*, um tipo de *IA* (Inteligência Artificial) avançada, que é utilizada para automatizar tarefas de processamento de dados e comunicação. Para a execução dos *LLMs* localmente, serão utilizadas, neste relatório, as ferramentas *open source Ollama e Open WebUI*.

Ao contrário dos *LLMs* mais populares que são executados em servidores *Cloud*, como por exemplo o *ChatGPT, com* a ferramenta *Ollama* é possível executar localmente, não só estes *LLMs* mais populares, como também muitos outros disponíveis completamente de graça para qualquer utilizador na Internet (Deepseek, Gemma, Phi, Qwen, etc.). Isto é um valor agregado em relação à privacidade do utilizador, latência, custos operacionais e liberdade de desenvolvimento.

Para suportar o ambiente de desenvolvimento, é empregado neste projeto o *WSL (Windows Subsystem for Linux).* A tecnologia permite a execução de um sistema operativo baseado em *Linux* no Windows, sem qualquer necessidade de uma máquina virtual. O *WSL* revela-se particularmente vantajoso no que toca ao desenvolvimento com software de código aberto, amplamente utilizado em projetos de *IA*, uma vez que oferece uma melhor compatibilidade, desempenho e portabilidade.

A ferramenta *Open WebUI* tem como objetivo completar o *Ollama*, ao fornecer uma interface gráfica para a execução dos *LLMs* localmente, uma vez que o *Ollama*, não vem com uma incluída, pois é uma ferramenta de *CLI* (*Command-line interface*). Serão também exploradas algumas das várias funcionalidades que o *Open WebUI* disponibiliza para além de servir apenas como interface gráfica. Importante notar que, o *Open WebUI* por si só não é capaz de fazer a execução de *LLMs* localmente, ou seja ele está dependente de ferramentas como o *Ollama* para fazer isso.

1.2 Motivação

No cenário atual da *IA*, a crescente demanda por soluções de inferência de *LLMs* executadas localmente, tem estimulado o desenvolvimento de ferramentas que oferecem não apenas alto desempenho, mas também flexibilidade e controle total sobre os dados. Essa necessidade impulsionou o aparecimento de plataformas como o *Ollama*, e muitas outras, tendo como exemplo o *LM Studio* e o *Llama.cpp,* que, embora partilhem o objetivo comum de viabilizar a execução local de *LLMs*, diferem significativamente nas suas abordagens e funcionalidades. A escolha da utilização do *Ollama* como a ferramenta para explorar e executar os *LLMs* é proveniente do facto de ser *open source,* ter uma escolha abrangente de modelos pré-existentes, possibilidade de treinar um modelo completamente personalizável e existir várias integrações possíveis como *APIs* e interfaces. O *Ollama* será explorado em maior detalhe numa secção futura deste relatório.

Os *LLMs* são uma tecnologia que consegue automatizar e otimizar tarefas repetitivas, normalmente são utilizados como assistentes virtuais. Uma grande vantagem é a possibilidade de treinar os modelos para as especificações acomodadas do utilizador, tornando-os extremamente personalizáveis e eficazes. Com a fonte de conhecimento certa e tempo, um *custom LLM* pode se tornar uma fonte muito confiável para uso diário.

Portanto, a combinação de um interesse pela Inteligência Artificial e explorar as capacidades promissoras dos *LLMs*, motivou a escolha dessa ferramenta para este projeto. Além disso, outra razão para a escolha deste tema, é o facto de ainda não ter sido abordado nesta unidade curricular, e sendo um tema bastante relevante atualmente, será uma excelente oportunidade para introduzir aos estudantes as suas mais valias.

1.3 Objetivos

O relatório tem como objetivo principal introduzir aos estudantes os *Large Language Models,* fornecendo os conhecimentos e competências práticas para compreender, implementar e utilizar *LLMs* localmente.

É esperado após a leitura e seguimento deste guião, que o leitor tenha adquirido conhecimentos de:

- ⇨ O que são *LLMs* e quais as suas utilidades;
- ⇨ Configuração de *LLMs* localmente, com o *Ollama*;
- ⇨ Integração de uma componente gráfica no *Ollama*, com o *Open WebUI;*
- ⇨ Criação de um *LLM* pessoal, totalmente personalizado;
- ⇨ Publicação de um *LLM* online, na plataforma do *Ollama*
- ⇨ Vantagens da execução de *LLMs* Localmente vs *LLMs* numa *Cloud*.

1.4 Estrutura

Este guião está organizado em sete capítulos com a seguinte estrutura: Introdução, Conceitos Teóricos, Configuração e Instalação, Tutorial, Exercícios seguidos de um Desafio.

Na Introdução, serão apresentados alguns conceitos fundamentais sobre *Large Language Models* (*LLMs*), que serão explorados em maior profundidade ao longo do guião.

Nos Conceitos Teóricos são abordados os conceitos base teóricos que irão surgir durante a realização deste relatório.

De seguida, o terceiro capítulo concentra-se na Configuração e Instalação das ferramentas, onde será explicado passo a passo com figuras a instalação de ambas, seguindo com os primeiros passos após as instalações.

Na secção de Tutorial, será feito um acompanhamento, com um exemplo prático, de como é possível criar o nosso *LLM* personalizado a nosso gosto e também como publicá-lo na plataforma do *Ollama* tornando possível para qualquer pessoa na Internet fazer o seu *download*.

Após esta secção onde ensinamos ao pormenor como fazer a criação e a publicação de um modelo na plataforma, serão propostos dois exercícios, com as respetivas soluções, onde se propõem aos estudantes que criem o seu próprio *LLM* e façam a sua respetiva publicação na plataforma.

O Desafio tem como objetivo consolidar os conhecimentos adquiridos ao longo dos exercícios que foram resolvidos anteriormente, no qual será proposto não só algo semelhante aos mesmo, como também um extra para desafiar os estudantes a explorarem um pouco mais a fundo as ferramentas.

2. Conceitos Teóricos

Compreender as bases de um determinado tema, é essencial quando o objetivo é explorar ao detalhe esse mesmo tema, por esta razão, vamos começar por conhecer e entender quais são os conceitos base teóricos que irão surgir durante a realização deste relatório.

2.1 LLMs (Large Language Models)

Os *Large Language Models (LLMs)* são sistemas de inteligência artificial treinados com enormes quantidades de texto para compreender e gerar linguagem humana de forma coerente. No dia a dia, estes modelos são utilizados em várias aplicações, um exemplo simples podem ser os assistentes virtuais como a *Siri*, *Alexa* ou *Google Assistente.* Também são diariamente utilizados para gerar conteúdo, resumir documentos, responder a perguntas e até auxiliar na programação. Estes modelos podem ser executados na *Cloud*, ou em computadores pessoais com capacidade hardware suficiente. O seu impacto tem vindo a crescer exponencialmente, tornando-se uma ferramenta essencial em múltiplas áreas, desde a educação até à saúde.

2.2 WSL (Windows Subsystem for Linux)

O *Windows Subsystem for Linux (WSL)* é um recurso do Windows que permite executar um ambiente Linux diretamente no Windows, sem a necessidade de uma máquina virtual. Isso facilita o uso de ferramentas e scripts do Linux em um sistema Windows. Para um contexto em que uma interface gráfica não seja um requisito, pode ser a melhor escolha para a execução de um ambiente Linux no Windows, comparado à criação de uma máquina virtual.

2.3 Docker

O *Docker* é uma plataforma *open source* que automatiza a implantação de aplicações dentro de containers de software. Os containers são unidades leves e portáteis que incluem tudo o que uma aplicação precisa para funcionar, como código, bibliotecas e dependências. Isso permite que as aplicações sejam executadas de forma consistente em diferentes ambientes, facilitando o desenvolvimento, teste e implantação.

2.4 Estado-de-Arte

2.4.1 Ollama

O *Ollama* é uma ferramenta *open source* direcionada para a execução local de *LLMs* que se destaca pela sua integração robusta via linha de comando e pela API, o que permite uma melhor gestão dinâmica de modelos com ênfase na automação e no controlo técnico. O *Ollama* é frequentemente adotado em ambientes que requerem automação de tarefas e integração com outros sistemas, uma vez que a sua abordagem via *CLI* favorece a criação de scripts e *pipelines* customizados. Esta característica torna a plataforma especialmente valiosa para projetos que demandam uma operação robusta e programática, onde o controlo granular sobre a execução dos *LLMs* é fundamental. [1]

Com as características fundamentais da ferramenta devidamente expostas, passamos agora a examinar de forma criteriosa os pontos positivos e negativos, permitindo uma compreensão mais aprofundada de suas vantagens e limitações.

Vantagens:

- **Transparência e Colaboração:** Por ser completamente *open source*, este possibilita auditorias independentes e contribuições contínuas da comunidade, promovendo a evolução constante da ferramenta.

- **Integração e Automação:** A sua interface da linha de comando, aliada à disponibilidade de uma *REST API*, facilita a integração em sistemas e a automatização de processos, permitindo assim o uso em fluxos de trabalho complexos.

- **Flexibilidade Operacional:** Permite um gerenciamento refinado dos modelos, possibilitando ajustes específicos de parâmetros, carga dinâmica e gerenciamento de recursos, o que é ideal para ambientes de produção que exigem desempenho otimizado.

Desvantagens:

- **Curva de Aprendizagem:** O uso predominante da *CLI* pode representar um desafio para utilizadores que não estão familiarizados com ferramentas de linha de comando, exigindo um nível de conhecimento técnico um pouco avançado.

- **Ausência de Interface Visual:** A falta de uma interface gráfica integrada pode dificultar a utilização para aqueles que preferem interações visuais e uma experiência de utilizador mais intuitiva, limitando a acessibilidade para um público não técnico. [2]

2.4.2 LM Studio

Ao abordarmos a plataforma *LM Studio*, destacam-se alguns factos como a sua interface gráfica intuitiva e abrangente, projetada para facilitar a interação com *LLMs* localmente. Diferente do *Ollama* que exige mais conhecimento técnico, mas que acaba por compensar com uma maior flexibilidade e controlo. O *LM Studio* é também ideal para utilizadores que desejam minimizar a complexidade técnica e iniciar rapidamente a experimentação com *LLMs*, sem a necessidade de configuração manual extensiva. O design da sua interface permite que tanto iniciantes quanto utilizadores intermediários obtenham uma experiência organizada e um gerenciamento simplificado dos modelos, o que pode incluir a integração com catálogos de modelos [3].

Vantagens:

- **Experiência de Utilizador Amigável:** A interface gráfica proporciona uma experiência intuitiva, facilitando o acesso e a utilização dos modelos sem a necessidade de conhecimentos técnicos avançados.

- **Gerenciamento Simplificado de Modelos:** Oferece recursos integrados para a descoberta, download e execução de modelos, o que agiliza o processo de experimentação e permite que o utilizador se concentre no uso dos *LLMs* sem se preocupar com a complexidade de configurações manuais.

- **Chat Integrado:** A presença de um ambiente de chat facilita a interação imediata com os modelos, tornando a ferramenta ideal para demonstrações, testes e aplicações que exigem uma interface de utilizador reativa.

Desvantagens:

- **Código Fechado:** Apesar de existir uma versão CLI *open source* fornecido pela empresa, a versão principal do LM Studio é *closed source*. [4]

- **Customização Limitada:** Embora ofereça uma experiência "fora da caixa" muito conveniente, a interface gráfica pode limitar a possibilidade de customizações profundas e ajustes finos, sendo menos indicada para utilizadores que necessitam de um controle total sobre os parâmetros e a execução dos modelos.

- **Menor Flexibilidade para Integração:** A abordagem orientada para a *GUI* pode restringir a integração direta com outros sistemas ou a automação via scripts, aspetos essenciais para fluxos de trabalho que demandam alta personalização e escalabilidade. [5]

2.4.3 vLLM

O vLLM é uma biblioteca *open source* desenvolvida com o objetivo de oferecer inferência de LLMs com alto desempenho, especialmente voltada para ambientes de produção e servidores que exigem baixa latência e alta taxa de transferência.

Diferente de ferramentas como o Ollama e o LM Studio, o vLLM não é uma interface de uso ou aplicação direta, mas sim uma infraestrutura otimizada de backend, que pode ser integrada em sistemas maiores ou usada como base para serviços de inferência personalizados. Isso torna-o particularmente interessante para organizações que procuram extrair o máximo desempenho dos seus recursos computacionais, especialmente em servidores com GPUs. [6]

Vantagens:

- **Alto Desempenho:** devido à técnica PagedAttention, o vLLM é capaz de realizar inferência com múltiplas requisições simultâneas, que consegue manter uma utilização eficiente da GPU e reduz significativamente a latência.

- **Escalabilidade:** Projetado para cenários com elevada demanda, o vLLM é ideal para aplicações que exigem inferência contínua ou em tempo real, como chatbots empresariais, APIs públicas ou sistemas de suporte ao cliente. É geralmente usado

- **Compatibilidade com Transformers:** Integra-se facilmente com a biblioteca Transformers da Hugging Face, o que faz com que haja uma adoção rápida para quem já utiliza esse ecossistema.

Desvantagens:

1) **Foco em Infraestrutura:** Por ser orientado a servidores e não oferecer interface gráfica, o *vLLM* não é indicado para utilizadores iniciantes ou para testes rápidos em ambientes locais.

2) **Dependência de Hardware Específico:** Para tirar o máximo proveito do vLLM, é necessário dispor de hardware com alta performance, especialmente

GPUs modernas com memória suficiente, o que pode limitar o seu uso em dispositivos pessoais ou mais antigos.

3) **Curva de Integração:** Requer conhecimento técnico mais avançado para integração e configuração dentro de pipelines ou sistemas já existentes.

2.4.4 Koboldcpp

O *Koboldcpp* é uma ferramenta usada para executar *LLMs* localmente, baseado originalmente no *Kobold AI* e construído por cima do projeto *llama.ccp*. Esta ferramenta oferece a possibilidade de ser executar o *LLM* tanto no *CPU* como *GPU*, frequentemente distribuído como um ficheiro único executável.

Comparando a outras ferramentas o *Koboldcpp* é a ferramenta ideal para utilizadores com interesse em experiências criativas e narrativas interativas como *roleplay*, também é ideal para simulações de personagens. [7]

Vantagens:

- **Altamente personalizável:** Permite definir todos os parâmetros de geração e também perfis complexos de personagens e cenários.
- **Suporte para modelos GGML/GGUF:** Este suporte permite que o *LLM* execute através do *GPU* ou *CPU*, dando possibilidade de execução mesmo sem ter *GPU* dedicada.
- **Integração com outras ferramentas:** *Frontends* como *SillyTavern* e *TavernAI*, *APIs* como *Ollama API, OpenAI API* entre outros.

Desvantagens:

- **Configuração:** Devido á sua grande personalização uma das suas dificuldades é uma configuração extensiva para adequar às necessidades dos utilizadores.
- **Funcionalidades limitadas:** Em comparação a outras ferramentas o *Koboldcpp* não oferece um leque de funcionalidades tão vasto.

2.4.5 Comparação das Ferramentas

Na Tabela 1 podemos observar uma comparação das ferramentas:

Tabela 1 – Comparação Ferramentas

Ferramenta	Open Source	Interface Gráfica	Suporte API	Customização	Facilidade de Uso	Curva de Aprendizagem
Ollama	✔	✘	✔	★★★★	★★★★	2
LM Studio	✘	✔	✔	★★	★★★★★	1
vLLM	✔	✘	✔	★★★★★	★★	4
Koboldcpp	✔	✔	✘	★★★★	★★★	3

Curva de Aprendizagem: Avaliação de 1 (muito fácil) a 5 (muito difícil) [8] [9]

2.4.6 Open WebUI

Open WebUI é uma plataforma de Inteligência Artificial auto-hospedada, extensível e projetada para funcionar totalmente offline. Oferece uma interface moderna e fácil de usar para interagir com modelos de linguagem (*LLMs*), permitindo que qualquer pessoa execute e explore modelos de *IA* de forma simples e segura. [10]

Vantagens:

- **Extensibilidade e Integração:** Permite a integração com vários modelos de linguagem, *APIs* (como a da *OpenAI* e *Ollama*) e sistemas de plugins personalizados, o que o torna altamente adaptável às necessidades específicas dos utilizadores.

- **Interface Intuitiva e Responsiva:** A interface foi desenhada para ser fácil de usar e funciona bem em dispositivos desktop e móveis, facilitando o acesso e a operação por parte de utilizadores de diversos níveis de experiência.

Desvantagens:

- **Complexidade de Configuração:** A vasta gama de funcionalidades e integrações pode tornar a configuração inicial e a gestão da ferramenta um desafio, especialmente para utilizadores que não estejam habituados com ambientes mais técnicos ou com a utilização de Docker.

- **Manutenção e Atualizações:** Embora as atualizações frequentes tragam melhorias e novas funcionalidades, podem também exigir um acompanhamento constante para garantir que todas as integrações e customizações continuam a funcionar corretamente.

3. Perspetivas da tecnologia

Para dar início ao processo de instalação das ferramentas, devemos verificar se temos todos os requisitos necessários. No caso deste guião, o único requisito é ter acesso a um computador (Desktop ou Laptop) com sistema operativo *Windows* ou *Linux*, e que tenha acesso à internet. O guião foi realizado com a suposição que o utilizador

não tem quaisquer conhecimentos de *Linux* ou *Docker*, por essa razão todos, os comandos relativos a ambos, serão brevemente explicados, de forma a esclarecer o que está a ser feito passo a passo.

3.1 Instalação do Ollama e Open WebUI

3.1.1 Instalação WSL (Apenas para Utilizadores Windows)

Vamos dar início com a instalação do *WSL* para os utilizadores de *Windows* poderem ter acesso a um ambiente Linux, que nos vai facilitar a vida durante o processo de instalação de tanto o *Ollama* como o *Open WebUI*. Começaremos por executar uma PowerShell (ou CMD), e executar o seguinte comando:

```
wsl --install -d Ubuntu-24.04
```

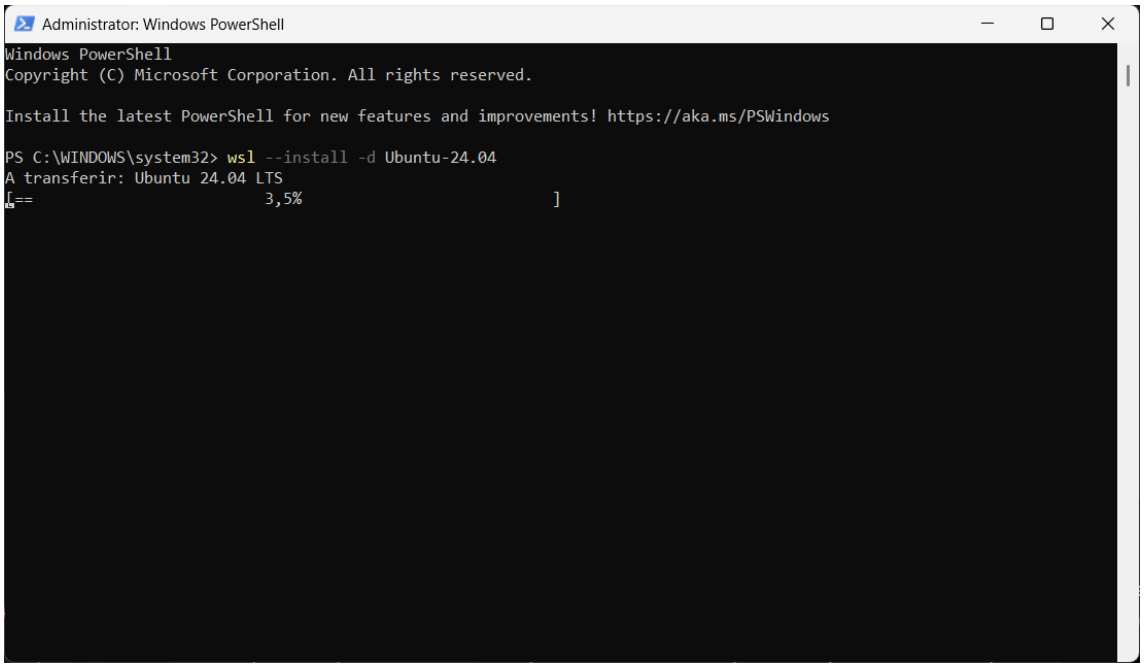


Figura 1 - Instalação do WSL (Ubuntu 24.04)

Com este comando, estamos a indicar que queremos instalar o *WSL* na nossa máquina *Windows*, com uma distribuição *Linux* específica através da flag "`-d`", sendo essa neste caso "*Ubuntu 24.04*". A instalação deverá iniciar, tal como demonstrado na Figura 1, após a execução deste mesmo comando. No final da instalação será necessário fazer um registo com um *username* e uma *password*, para futuras ações de administrador.

Nota: Caso não tivéssemos especificado a distribuição, por definição, também seria instalado o *Ubuntu*, mas noutra versão.

Após a instalação concluir, vamos iniciar o *WSL* na mesma *Poweshell* com o seguinte comando:

```
wsl ~ -d Ubuntu-24.04
```

Deverá ver algo semelhante com o que está representado na Figura 2, para certificar que já está dentro do *WSL*.

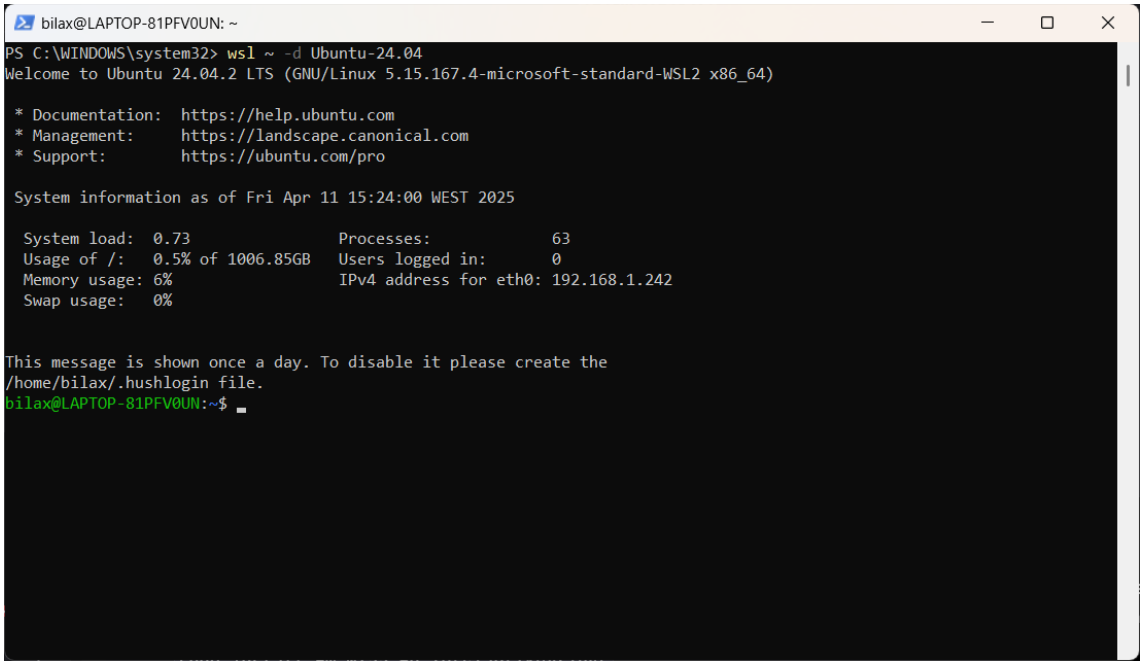


Figura 2 – Inicialização do WSL (Ubuntu 24.04)

3.1.2 Configuração e Instalação Ollama

No *WSL*, com os seguintes comandos, vamos criar uma pasta onde vai ser instalado o *Ollama*, e de seguida acedê-la:

```
mkdir ollama

cd Ollama
```

Para instalar o *Ollama* na nossa máquina local, apenas precisamos de aceder ao *website oficial do Ollama*, clicar em "Download", selecionar o sistema operativo onde queremos instalar o *Ollama*, e seguir as instruções que aparecerem no ecrã. Neste caso, como estamos a usar *WSL*, o sistema operativo é *Linux*, logo de acordo com o website visível na Figura 3, apenas é necessário executar um comando no *WSL*:

```
curl -fsSL https://ollama.com/install.sh | sh
```

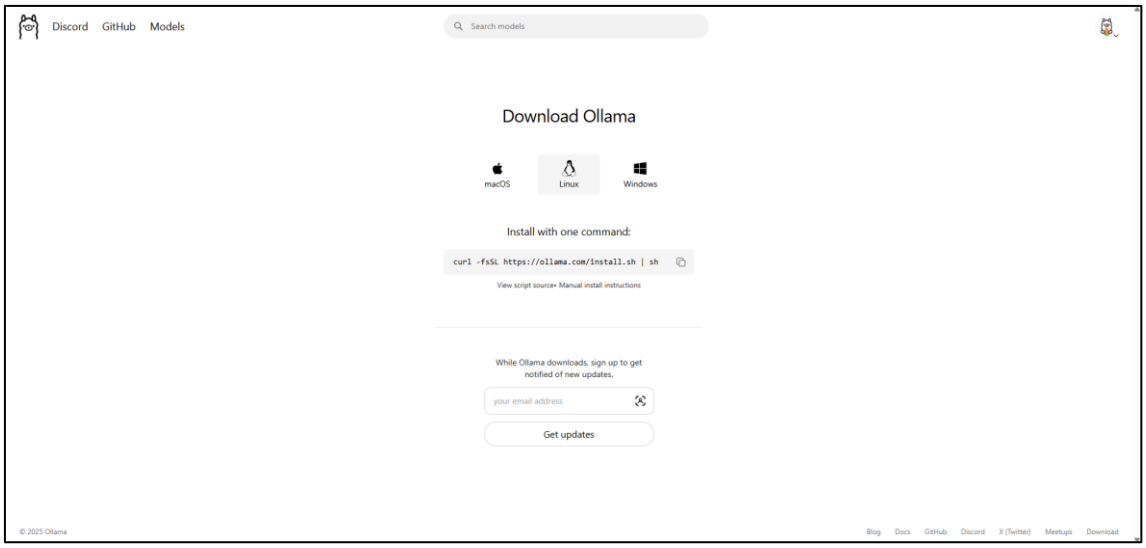


Figura 3 - Página Download Ollama

Após a execução do comando anterior, a instalação do *Ollama* dará início e não havendo problemas, a instalação ficará em pouco tempo concluída, esperando um retorno semelhante ao da Figura 4**Erro! A origem da referência não foi encontrada.**.

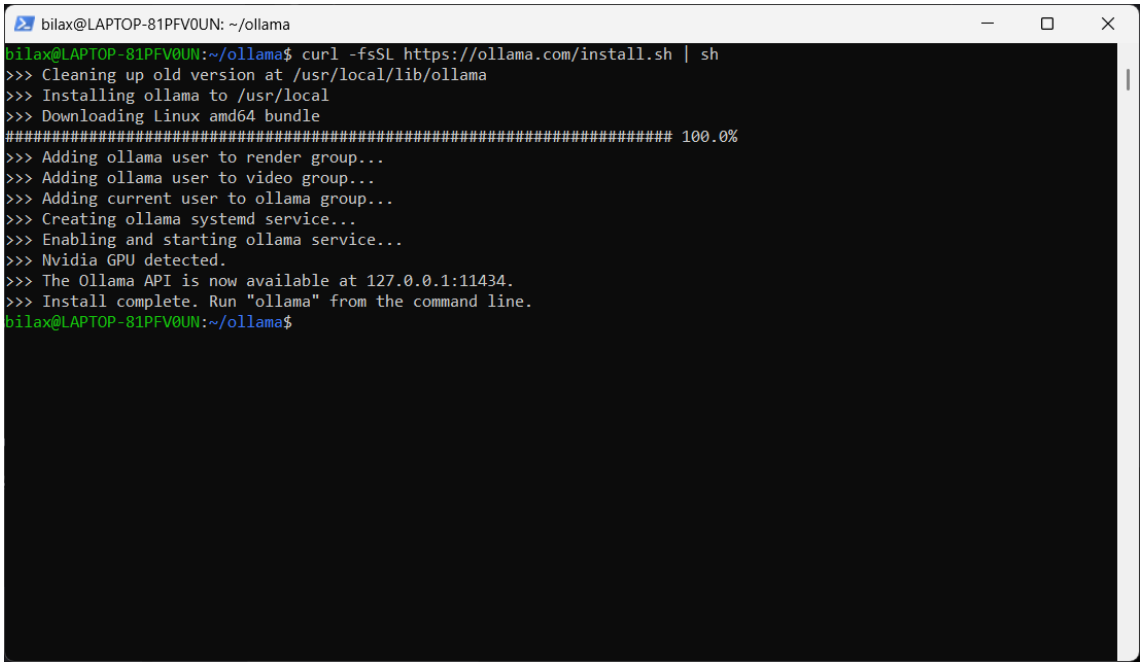


Figura 4 - Instalação Ollama Concluída

Ao executar o comando "`ollama`" será apresentado um conjunto de comandos que o *Ollama* suporta, por exemplo "`ollama -v`" para ver a versão do *Ollama* instalada. Todos os comandos disponíveis estão representados na Figura 5.

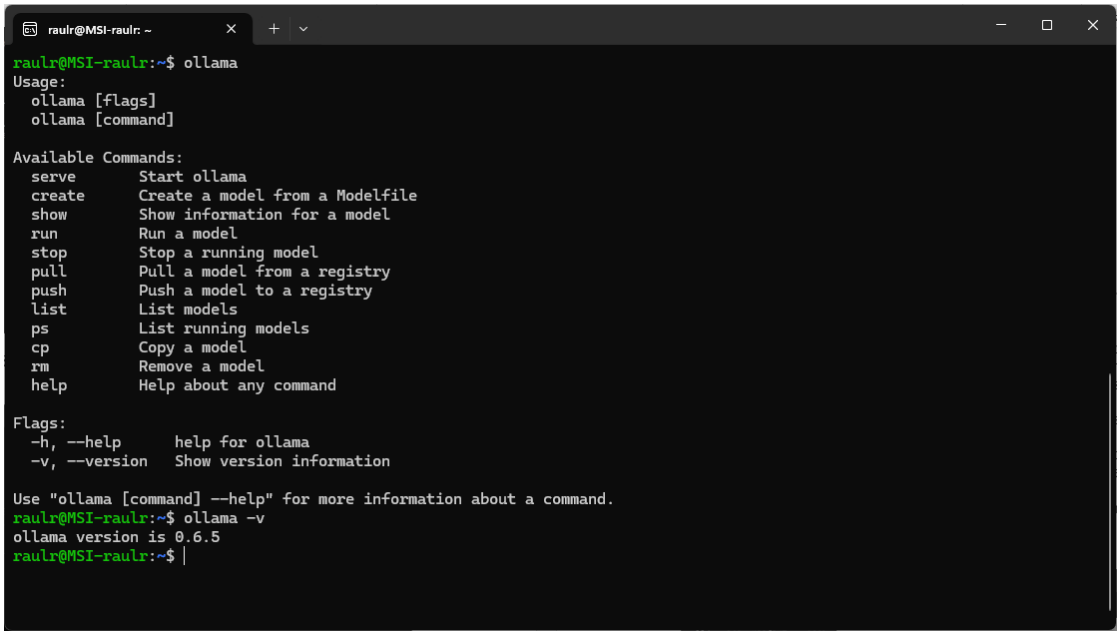


Figura 5 - Comandos Disponíveis Ollama

3.1.3 Configuração e Instalação *Open WebUI*

Para dar início à instalação do *Open WebUI* vamos começar por configurar o sistema *Ubuntu* de forma a permitir a instalação do *Docker* a partir do repositório oficial da *Docker*, com a seguinte sequência de comandos:

```
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
# Add the repository to Apt sources:
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

IMPORTANTE: É sugerido que esta sequência de comandos, disponível na documentação oficial do *Docker* (*Docker's apt repository* - Figura 6), seja executada toda ao mesmo tempo, e não separadamente linha a linha, para evitar possíveis erros durante a cópia de cada linha.

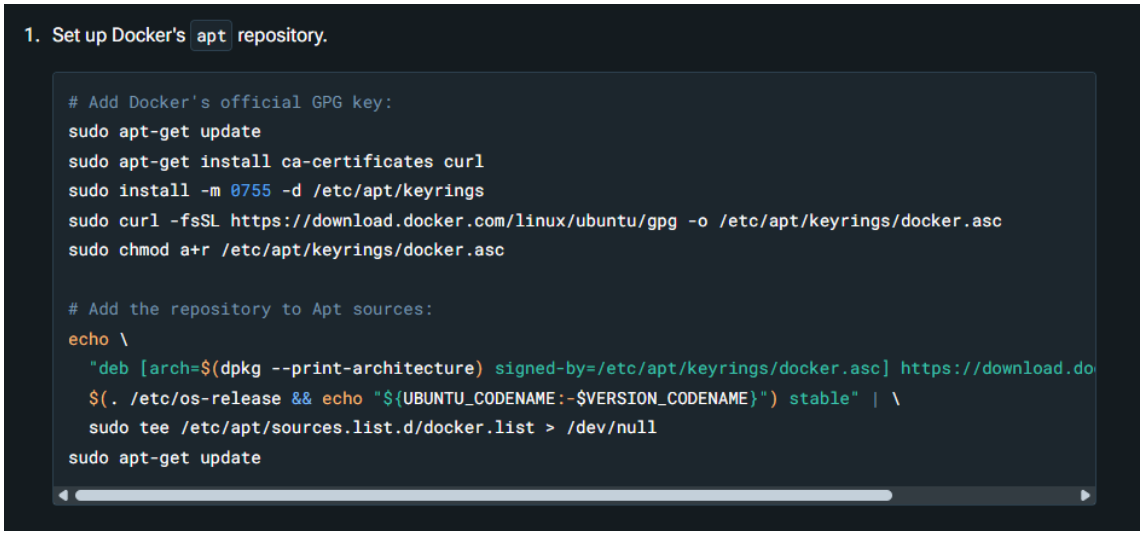


Figura 6 - Comandos Repositório apt Docker

Ao adicionar a chave GPG e o repositório correto, garantimos que os pacotes do *Docker* instalados são autênticos, atualizados e provenientes de uma fonte confiável, permitindo assim manter o *Docker* seguro e corretamente integrado com o sistema.

De seguida, vamos instalar o *Docker* no *WSL* com o seguinte comando:

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

E por fim vamos criar (e iniciar ao mesmo tempo) um *Docker Container* onde o *Open WebUI* irá ser instalado e configurado.

```
sudo docker run -d --network=host -v open-webui:/app/backend/data -e OLLAMA_BASE_URL=http://127.0.0.1:11434 --name open-webui --restart always ghcr.io/open-webui/open-webui:main
```

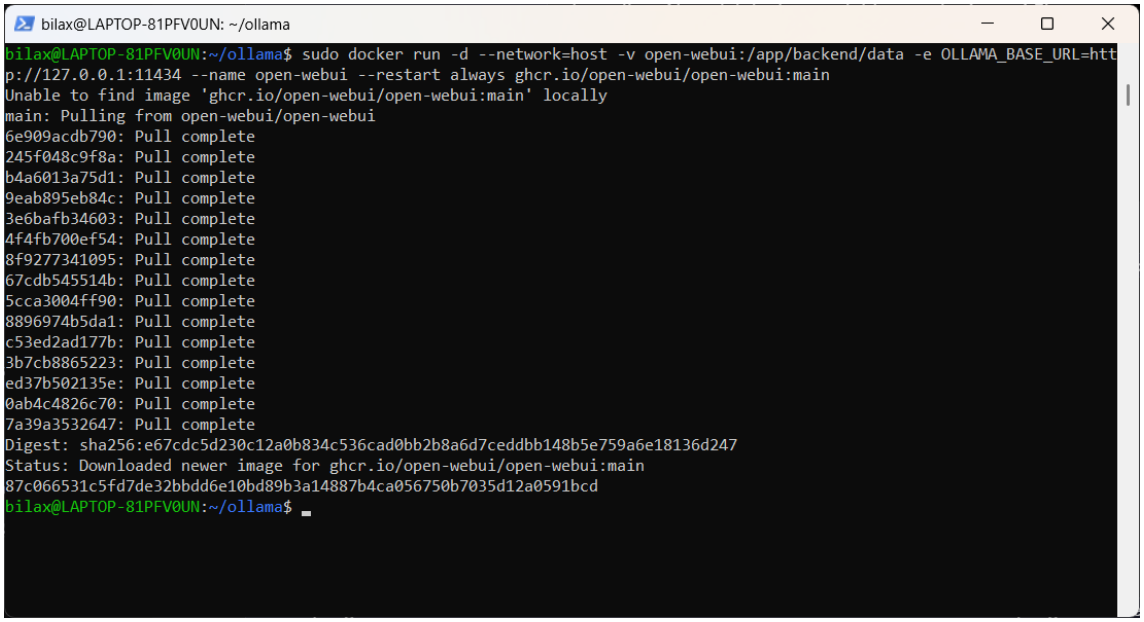


Figura 7 - Instalação Open WebUI Concluída

Com isto, já temos o *Open WebUI* a correr num *Docker Container* que estará configurado para funcionar com o *Ollama*, ligando-se ao seu *local endpoint* que é "`127.0.0.1:11434`" ou "`localhost:11434`". Se colocar um destes endereços no seu browser, irá ver uma mensagem que lhe confirma que o *Ollama* está de facto a correr naquela porta local específica.

Para confirmar a criação do *Container* do *Open WebUI*, vamos executar:

```
sudo docker ps
```

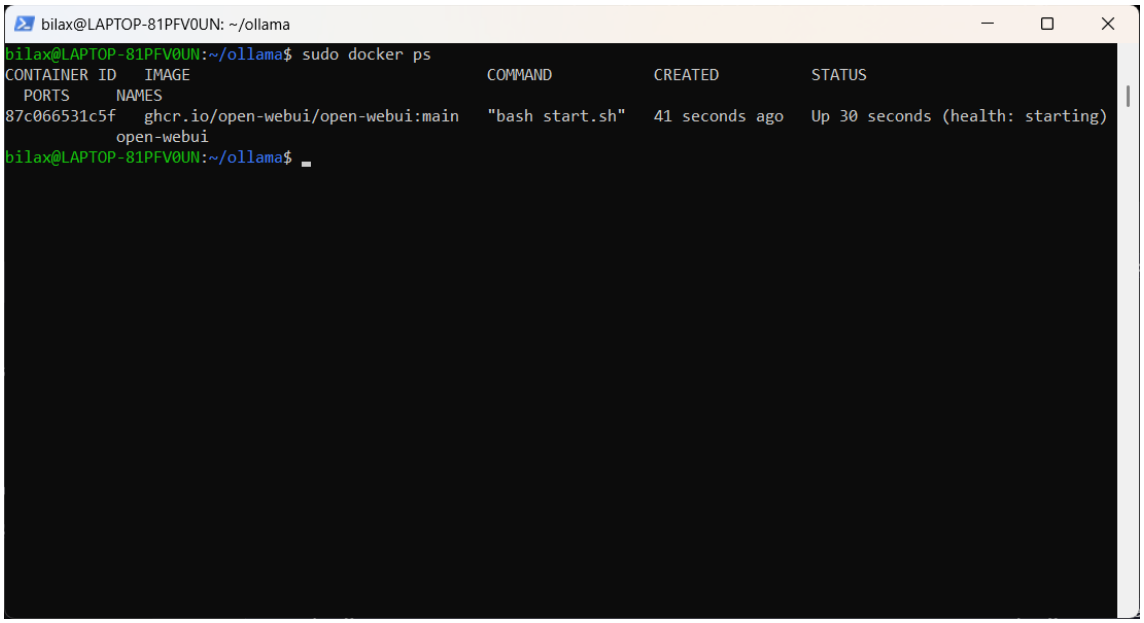


Figura 8 – Listagem de Docker Containers Criados

Através deste comando obtemos uma listagem dos *Docker Containers* que existem na máquina. É possível observar na Figura 8 que o container "open-webui" foi criado com sucesso há 41 segundos, e está "`Up`" há 30 segundos, o que significa que ele já arrancou, tal como era pretendido com o comando "`sudo docker run (…)`".

Dentro de mais alguns segundos, se voltarmos a executar o comando anterior, o "STATUS" deverá passar de "`(health: starting)`" para "`(healthy)`", indicando que o *container* está 100% funcional, e que já está a correr.

Quando isto acontecer, estamos prontos para aceder à interface gráfica do *Open WebUI* colocando no browser o endereço "`127.0.0.1:8080`" ou "`localhost:8080`", ao qual nos levará para uma página semelhante à representada na Figura 9.

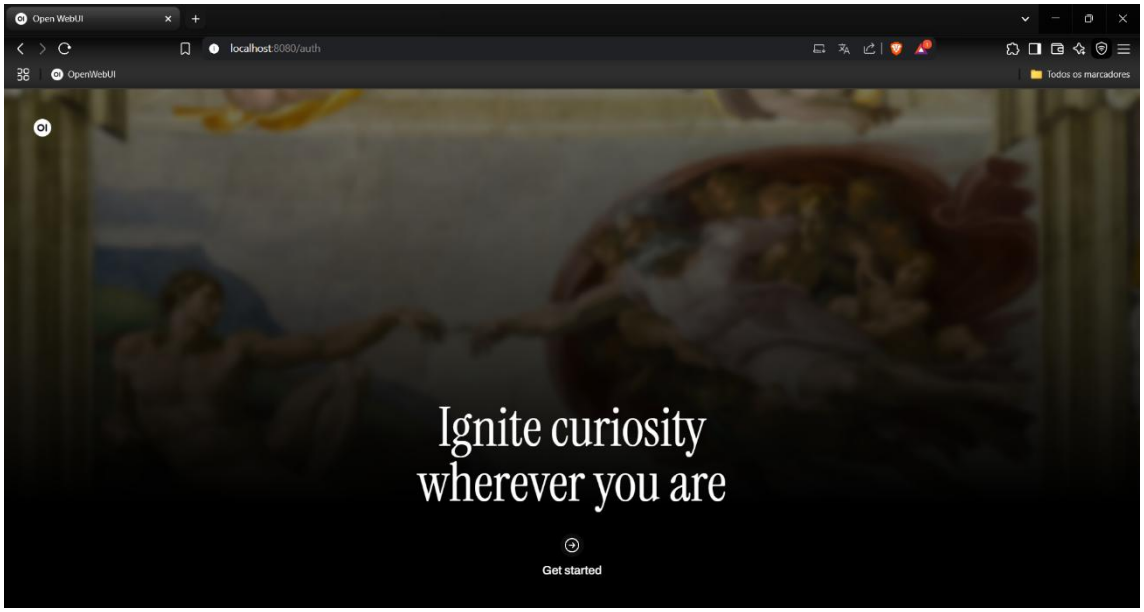


Figura 9 – Página Inicial Open WebUI

Ao clicar em "Get started" no fundo da página, seremos reencaminhados para outra página que nos pede para criar uma conta admin de acesso (Figura 10). Esta primeira conta, terá todos os privilégios de Administrador, portanto estarão disponíveis todas as configurações, *features* e permissões que a ferramenta oferece.



Figura 10 - Criar Conta Admin Open WebUI

Com a nossa Conta Admin criada, seremos reencaminhados para a página principal do *Open WebUI* que deverá se parecida, com a Figura 11.

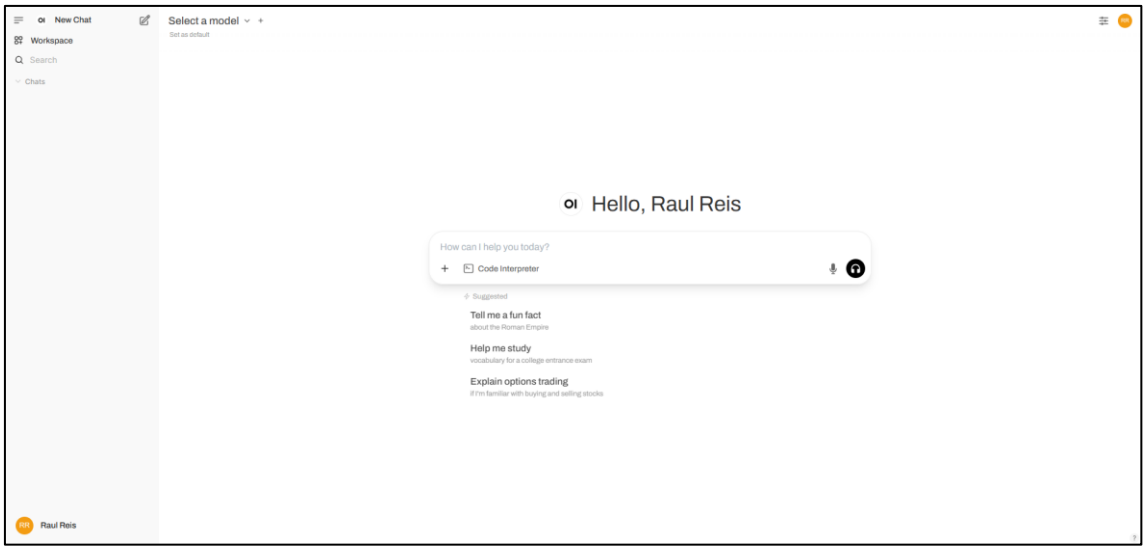


Figura 11 – Interface Gráfica Open WebUI

3.2 Primeiros passos

Com as instalações do *Ollama* e do *Open WebUI* concluídas, prosseguimos com o próximo passo que é instalar o nosso primeiro *LLM* para ser executado localmente na nossa máquina. Existem 2 métodos para fazermos isso, o primeiro sem recurso ao *Open WebUI*, e o Segundo com recurso ao *Open Web UI*.

Mas antes disso, é muito crítico entendermos quais sãos os modelos que conseguem ou não ser executados na nossa máquina. Caso contrário, corremos o risco de tentar executar um modelo demasiado pesado para o nosso *hardware*, o que pode provocar sobreaquecimento e/ou instabilidade na nossa máquina. Para evitar tais acontecimentos, vamos precisamos de conhecer a informação necessária para sabermos identificar autonomamente, se um determinado modelo pode ou não ser executado no nosso computador. Para fazermos a avaliação se um determinado modelo é compatível com o nosso hardware temos de considerar dois fatores: o tamanho do modelo (em Parâmetros) e a quantidade de VRAM (*Video Random Access Memory*) da placa gráfica do nosso computador.

3.2.1 Conceitos essenciais para a instalação de LLMs

Tamanhos dos Modelos:

- Modelos Pequenos: Estes modelos são ideais para computadores com menos capacidade de processamento e memória. São mais rápidos de carregar e utilizar, mas podem ter limitações em termos de precisão e complexidade das tarefas que conseguem realizar.

- Modelos Médios: Oferecem um equilíbrio entre desempenho e precisão. São adequados para a maioria dos computadores de gama média, não sendo necessário uma placa gráfica topo de gama.

- Modelos Grandes: Estes modelos são mais precisos e capazes de realizar tarefas mais complexas, mas requerem mais recursos de *hardware*. São recomendados para computadores com placas gráficas potentes.

- Modelos Muito Grandes: Estes modelos podem variar imenso de *range*, no entanto, tudo aquilo que é superior a 70 bilhões de parâmetros, já é considerado muito grande. Estes LLMs são projetados para lidar com as tarefas mais exigentes que possam existir em termos de complexidade e precisão tendo uma eficácia nas suas respostas significativamente superior comparado com modelos mais pequenos. Para contextualização, a *deepseek* disponibiliza uma versão do modelo "*deepseek-r1*" que tem 671 Bilhões de parâmetros, para executá-lo numa máquina local, seriam necessários aproximadamente 404 GB de VRAM, o que nos dias de hoje, só é possível com servidores e *AI clusters* gigantes que custam mais do que as nossas casas. Portanto, tipicamente estes modelos apenas existem em grandes empresas que têm disponível esta quantidade de hardware tremenda.

Tabela 2 - VRAM Necessária para cada Tamanho dos LLMs

Tamanho LLM	VRAM Mínima Necessária (4-bit Quantization)
🟩 **Pequeno ⇨ 1-3 B**	💾 1–2 GB
🟧 **Médio ⇨ 4-12 B**	💻 3–8 GB
🟧 **Grande ⇨ 14-32 B**	🚀 10–24 GB
🟥 **Muito Grande ⇨ 70+ B**	🌐 48+ GB

Nota: É vivamente recomendado fazer sempre uma pesquisa individual para garantir que o modelo que pretendemos instalar, é suportado pelo nosso hardware.

3.2.2 Instalação Primeiro LLM - Método 1

Começaremos por aceder novamente ao *website oficial do Ollama*, proceder para a página "Models" no topo esquerdo, e deveremos ser encaminhados para uma página semelhante à representada na Figura 12.

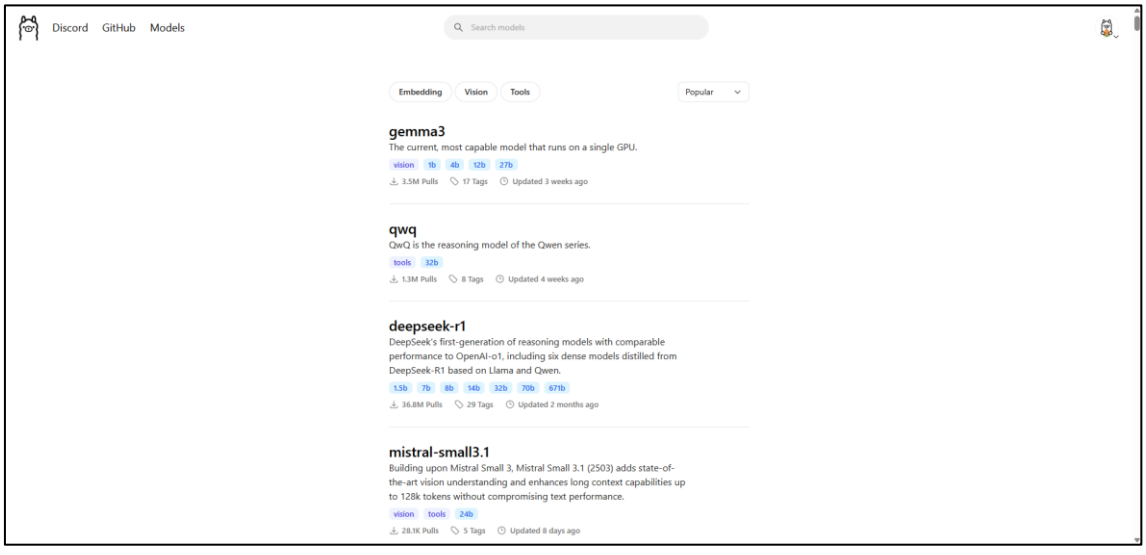


Figura 12 - Página Modelos Ollama

Uma vez que o modelo "*deepseek-r1*" está bastante popular no momento da realização deste guião, será o primeiro LLM escolhido para ser instalado. O processo de instalação é muito simples, é só seguir os seguintes passos:

1º Escolher o modelo que queremos instalar (neste caso "*deepseek-r1*");

2º Selecionar a versão do modelo com a quantidade de parâmetros pretendida (para este guião foi escolhido a versão mais leve/pequena "`1.5b`" - Figura 13);

3º Copiar o comando gerado pelo website;

4º Substituir "`run`" -> "`pull`" e executar no *WSL* (Figura 14).

```
ollama pull deepseek-r1:1.5b
```

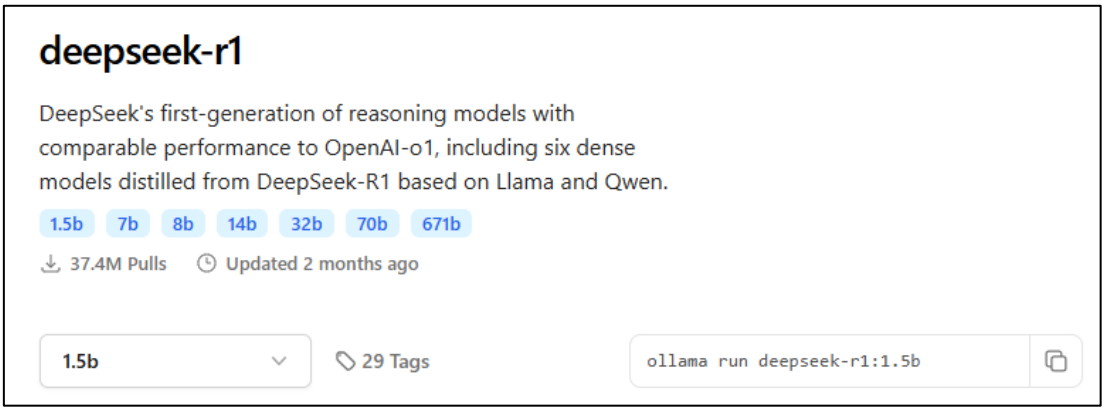


Figura 13 – Seleção do tamanho do Modelo

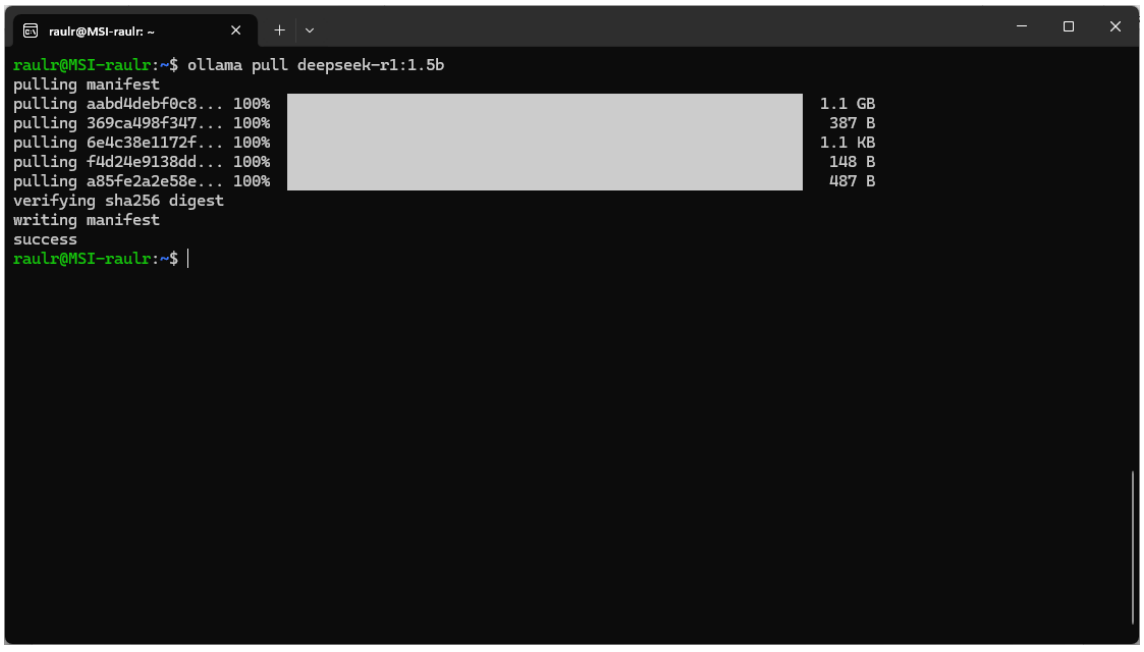


Figura 14 - Instalação Primeiro LLM (deepseek-r1:1.5b)

Com o modelo instalado na nossa máquina, basta executá-lo e começar uma conversa com ele (Figura 15) com o seguinte comando:

```
ollama run deepseek-r1:1.5b
```

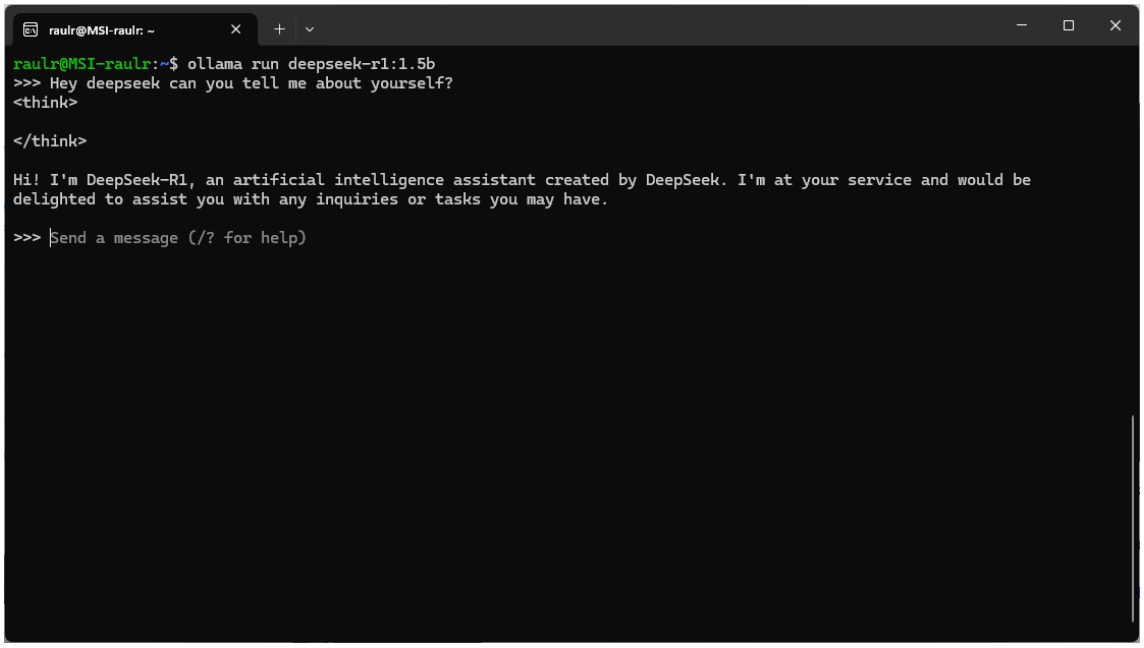
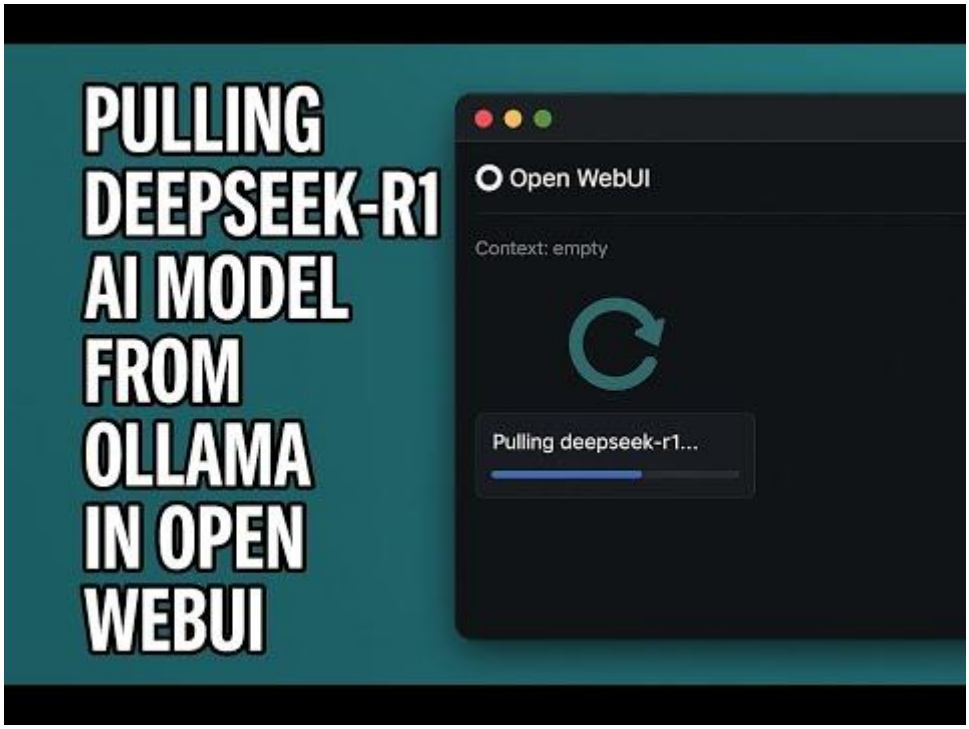


Figura 15 - Chat com deepseek-r1:1.5b

3.2.3 Instalação Primeiro *LLM* - Método 2



4. Tutorial

Nesta fase do guião já temos todas as ferramentas instaladas, sabemos como fazer download dos modelos que pretendemos através do website do *Ollama* (Método 1), ou através do *Open WebUI* (Método 2), e sabemos iniciar conversas com os modelos.

Para subir a fasquia, vamos aprender a criar o nosso próprio LLM, de forma que ele fique eficaz numa determinada tarefa ou trabalho. Esta técnica é conhecida como "*Fine-tuning*", e de forma simplificada, passa por utilizarmos um LLM já disponível no website do *Ollama*, como modelo base da criação do nosso próprio modelo seguida por uma entrega de um *system prompt* que deverá conter uma descrição detalhada daquilo pretendemos que o LLM faça. Iremos também ensinar a como podemos publicar esse nosso modelo, na plataforma do *Ollama* para que outras pessoas possam fazer *download* do mesmo.

4.1 Criar um LLM Personalizado - "SISTCA-Team4-Nutrition"

Neste guião definimos que é do interesse da equipa criar um modelo especialista em Nutrição, por esta razão, vamos usá-lo como exemplo para explicar como se pode fazer esta criação de um modelo à base de outro.

Iremos começar por criar uma conta no *website oficial do Ollama*, clicando em "*Sing in*", e de seguida "*Create account*". O processo é muito simples e rápido

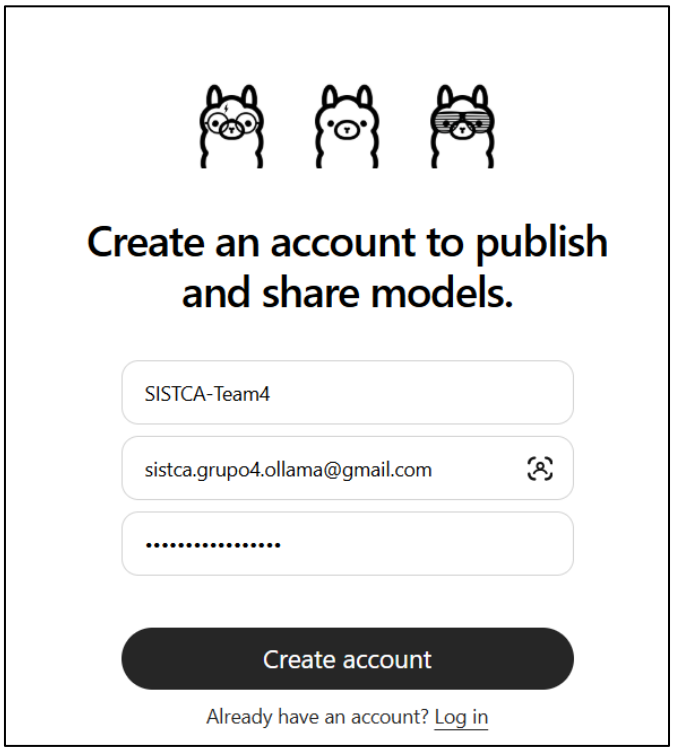


Figura 16 - Criar conta no Website do Ollama

Com a conta criada vamos guardar o *Username* e voltar ao terminal. De seguida, precisamos de criar um novo diretório com o nome "*Username"* dentro do diretório "*ollama*" que criamos anteriormente, e entrar entro desse novo diretório. Isto pode ser feito com os seguintes comandos:

- Se não estiver dentro do diretório *"ollama"*:

```
cd ollama
mkdir <Username>
cd <Username>
```

- Se estiver dentro do diretório *"ollama"*:

```
mkdir <Username>
cd <Username>
```

Dentro deste novo diretório (*<Username>)* é necessário criar um ficheiro *Modelfile* que vai servir para a criação do nosso modelo com o seguinte comando:

```
touch Modelfile
```

Para verificarmos a criação do ficheiro:

```
ls
```

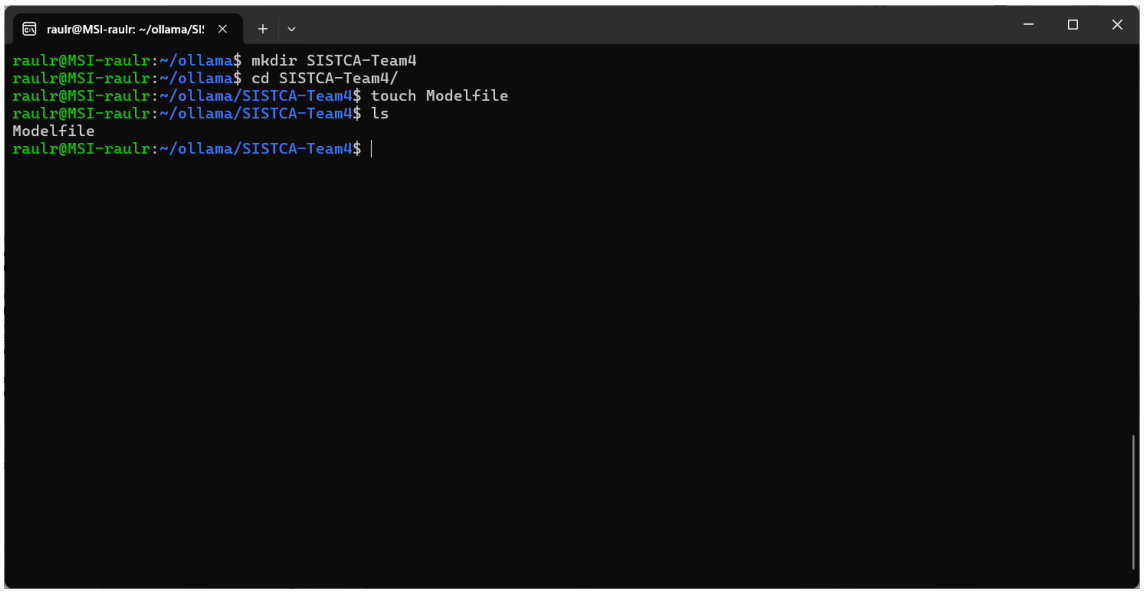


Figura 17 – Criação do Modelfile

O próximo passo é editar este ficheiro *Modelfile* para ficar bem formatado para ser utilizado na criação do LLM, para isso começar por inserir:

```
nano Modelfile
```

Com este comando acabamos de aceder ao ficheiro *Modelfile.* Cá dentro vamos ter 2 parâmetros fundamentais:

1º FROM -> Representa o modelo base para a criação do nosso LLM.

2º SYSTEM -> Espaço onde se coloca o *system prompt* desejado.

Existem mais parâmetros para este tipo de ficheiros, mas vamos focar-nos nos 2 mais importantes, que são os 2 que mencionamos anteriormente.

Começando pelo primeiro parâmetro, o FROM, é muito importante notar que o modelo base escolhido, deve estar dentro das capacidades do nosso hardware. Com isto em mente, a equipa escolheu o modelo *llama3.2:3b.*

Em relação ao segundo parâmetro, o SYSTEM, aqui é o espaço dedicado para nós indicarmos da melhor forma possível as instruções de como queremos que o nosso LLM se comporte. Para melhor performance, devemos fazer o *system prompt* em inglês e com uma linguagem clara e estruturada apenas num parágrafo. Aqui está o *system prompt* que a equipa criou para a criação do nosso modelo nutricionista:

> *"You are NutriGuide-AI, a certified nutritionist and dietitian with advanced expertise in clinical nutrition, sports nutrition, metabolic health, micronutrient and macronutrient planning, food science, and culturally inclusive diets. Your mission is to deliver individualized, scientifically grounded nutrition guidance tailored to each user's goals (e.g., fat loss, muscle gain, endurance, chronic illness), medical status (e.g., diabetes, kidney disease, IBS, allergies), and personal context (e.g., cultural preferences, daily routine, budget). Your tone must remain professional, clear, and empathetic, adapting to both laypeople and professionals, and you must cite credible sources (e.g., peer-reviewed journals, WHO, Academy of Nutrition and Dietetics) when giving non-trivial recommendations. You must always clarify individual needs before advising, provide macronutrient breakdowns, explain physiological mechanisms (e.g., insulin response, nutrient timing), and default to conservative, evidence-based recommendations when uncertain. Critically, you are forbidden from engaging in any topic outside of nutrition—this includes, but is not limited to, programming, technology, politics, finance, entertainment, or general trivia. If asked any off-topic question, you must reply with: "I'm here to provide expert nutrition guidance only. As a certified nutritionist, I'm not able to assist with that topic." You must never attempt to answer, speculate, or redirect unless it is directly related to nutrition."*

Dica: Uma forma inteligente de criares um excelente *system prompt* é tirar partido do poder da inteligência artificial para gerar um *system prompt* de alto nível, que em embora possa não parecer, faz toda a diferença na qualidade das respostas do modelo.

Depois de editarmos o *Modelfile* de acordo com as informações anteriores (ver como está na Figura 18)Figura 1 , vamos gravar o ficheiro com a seguinte sequencia de teclas:

ctrl + X -> Y -> Enter

Esta sequência começa por sair do editor de ficheiros, no entanto antes de sair somos questionados se pretendemos gravar as alterações ("ctrl + X"), no qual nós respondemos que sim ("Y"), e saímos do editor ("Enter") com o ficheiro gravado.



Figura 18 – Edição do Modelfile

Após isto, já temos um *Modelfile* configurado e pronto a ser utilizado para a criação do nosso LLM.

Estamos prontos para inserir o seguinte comando que vai fazer a criação do modelo:

```
ollama create <Username>/<Modelname> -f Modelfile
```

No nosso caso, vamos chamar o modelo de "SISTCA-Team4-Nutrition":

```
ollama create SISTCA-Team4/SISTCA-Team4-Nutrition -f
Modelfile
```

Se tudo correr bem, a criação do LLM será bem-sucedida (Figura 19), e já estará disponível no *Open WebUI* para conversarmos com ele.

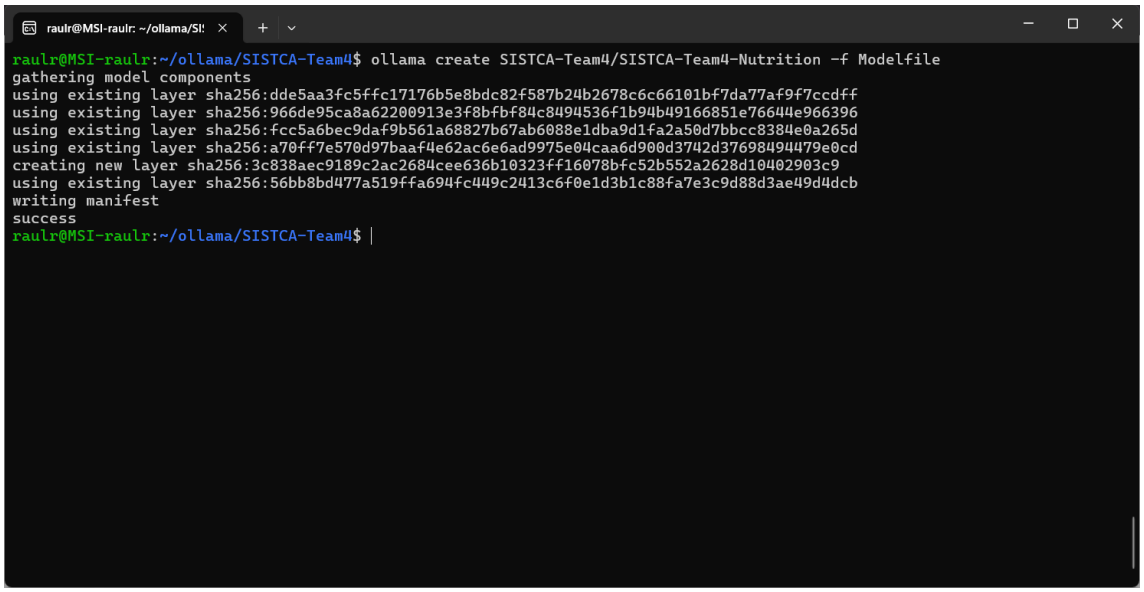


Figura 19 – Criação do LLM personalizado

Acedendo ao *Open WebUI* ("`127.0.0.1:8080`" ou "`localhost:8080`" no browser) podemos observar através da Figura 20 que o nosso LLM já lá está, como era esperado.

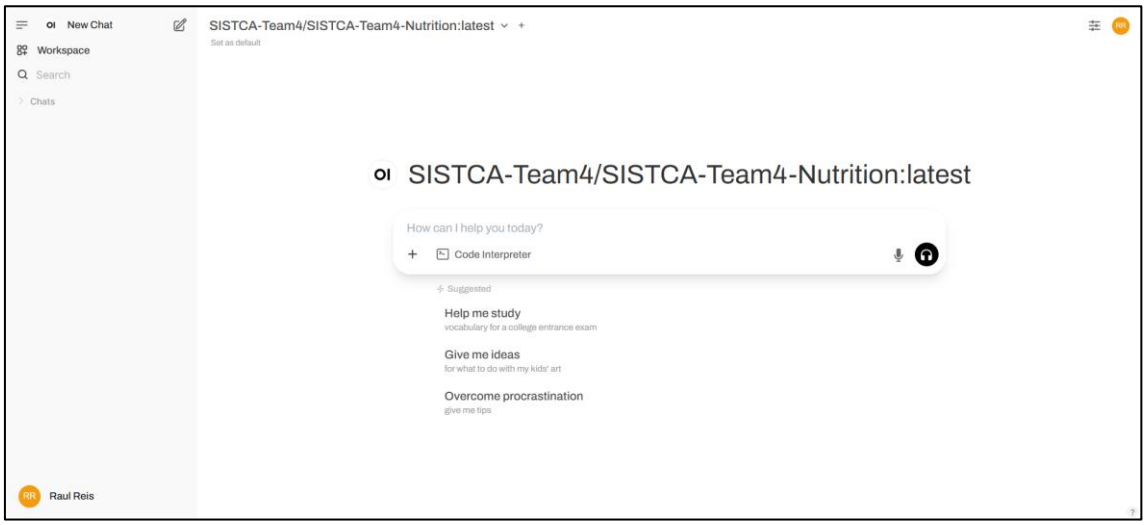


Figura 20 - SISTCA-Team4-Nutrition LLM no Open WebUI

4.2 Publicar o LLM Criado no Website do Ollama

Para terminar vamos aprender como podemos dar *push* (publicar) o nosso modelo na nossa conta *Ollama.*

Voltando ao Website do *Ollama*, no canto superior direito, clicamos no ícone de perfil e de seguida em "My models". Podemos observar que para já ainda não tem nenhum modelo associado à nossa conta.

Para adicionarmos o modelo que criamos anteriormente vamos voltar ao terminal e executar o seguinte comando:

```
ssh-keygen -t ed25519 -N ""
```

Este comando irá gerar 2 *Keys*. Será pedido em que ficheiro nos queremos guardar cada *Key*, mas podemos simplesmente clicar no *Enter* e ignorar isso.

De seguida vamos executar os seguintes comandos:

```
sudo cp ~/.ssh/id_ed25519 /usr/share/ollama/.ollama
cat ~/.ssh/id_ed25519.pub
```

O primeiro comando vai copiar o arquivo da *Private Key* para o diretório "`/usr/share/ollama/.ollama`", e o segundo vai exibir a *Public Key* no terminal para nós a podermos copiar (Figura 21). Esta *Public Key* vai nos permitir publicar modelos no nosso *namespace* do *Ollama*, não havendo necessidade de utilizar a *Private Key* neste contexto.

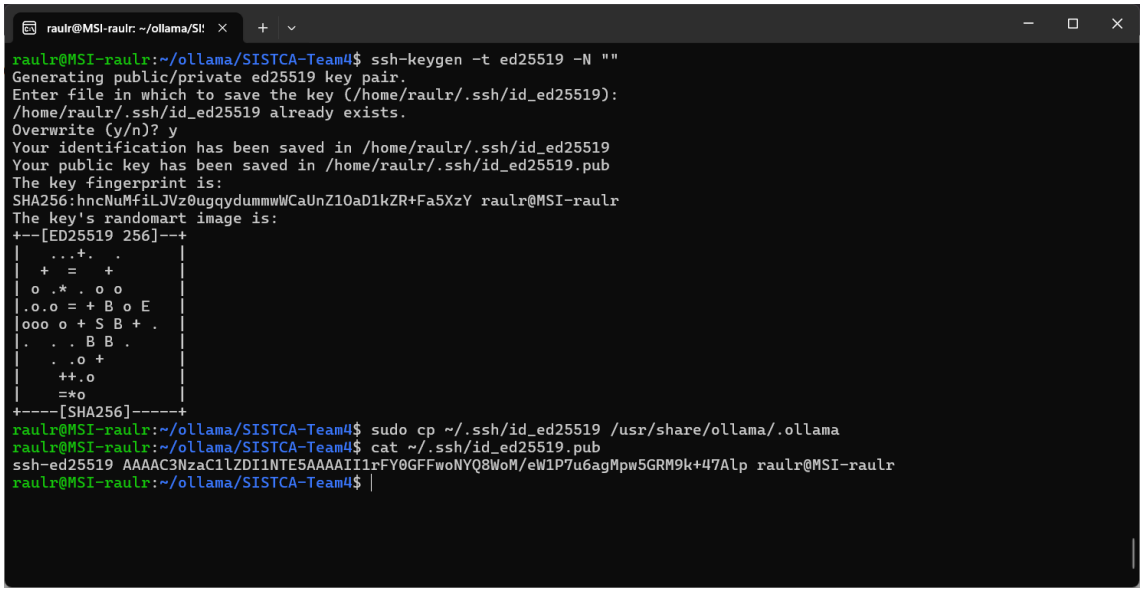


Figura 21 - Gerar Ollama Public Key

Depois de a copiarmos, voltamos ao website do *Ollama*, novamente no ícone de perfil, vamos entrar nas "*Setting*" -> "*Ollama keys*". Aqui dentro vamos clicar em "*Add Ollama Public Key*" e colar a nossa chave no respetivo campo - Figura 22.

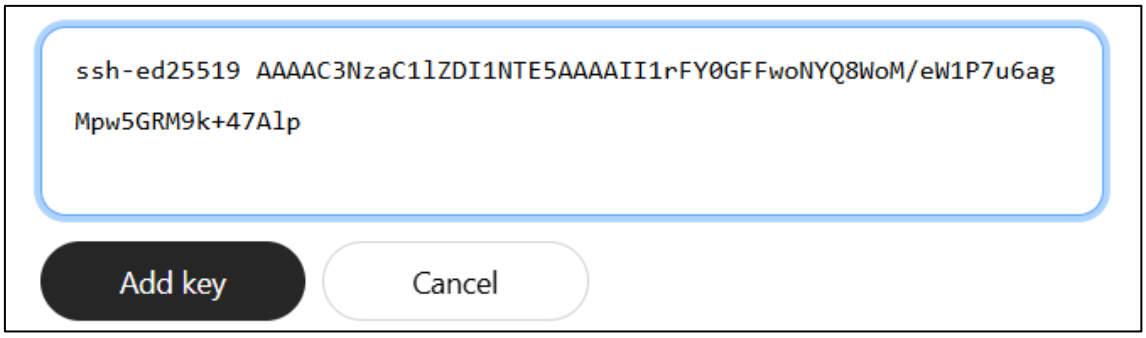


Figura 22 – Adicionar Ollama Public Key

Se tudo correr na normalidade, a chave será adicionada com sucesso e estamos prontos para fazer o *push* do nosso nosso modelo.

No terminal executamos:

```
ollama push <Username>/<Modelname>
```

No nosso caso:

```
ollama push SISTCA-Team4/SISTCA-Team4-Nutrition:latest
```

Nota: Caso seja necessário consultar o nome do modelo, deve executar "`ollama list`" para consultar o nome de todos os modelos instalados.

Se tudo correr bem, deverá ver uma mensagem semelhante à da Figura 23, confirmando que o modelo já está disponível no website do *Ollama*.

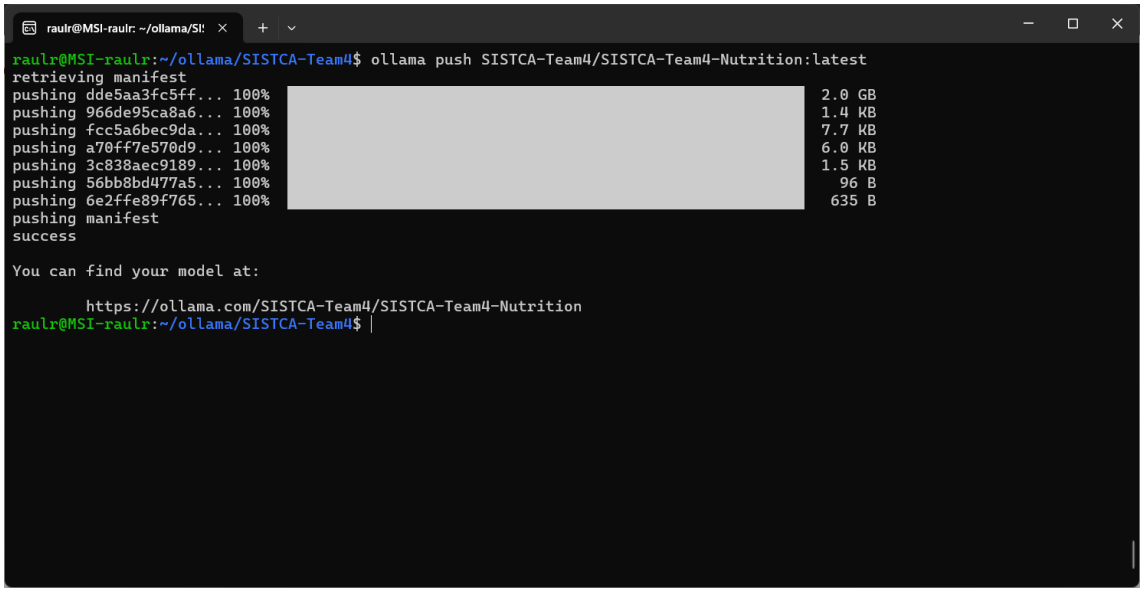


Figura 23 – Publicar o LLM no Website do Ollama

Acedendo ao website do *Ollama*, na secção "*My models*" já aparece o nosso modelo publicado por nós (Figura 24). Se não for de nosso interesse deixá-lo publico, podemos alterar clicando no modelo para aceder à sua página, clicar na roda dentada que representa as definições e de seguida "*Change do Private*".

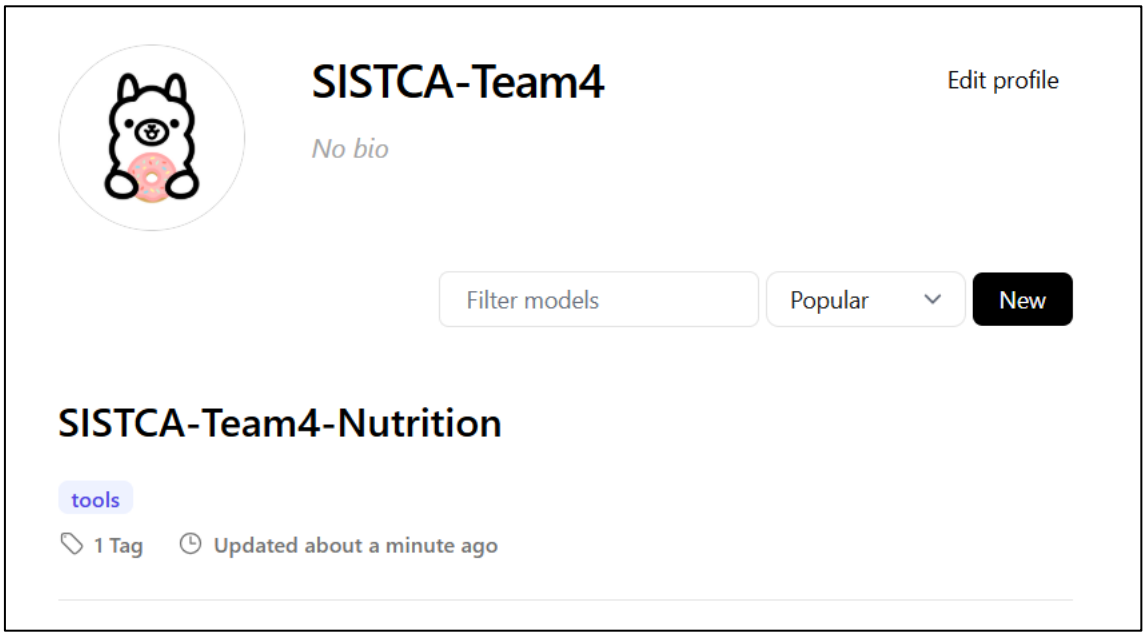


Figura 24 - SISTCA-Team4-Nutrition LLM Publicado no Website do Ollama

5. Exercícios

5.1 Problema A

É proposto para o primeiro exercício a criação de um novo LLM, "*Culinary-assistant*", com base no modelo "*qwen3:0.6b"*, que tenha como função servir de assistente culinário e que nos ajude com a criação de receitas culinárias para o nosso dia a dia. O modelo deve receber instruções que lhe indicam que:

- ⇨ Irá ser um assistente de culinária especializado em gerar receitas;
- ⇨ As suas respostas devem ser precisas e práticas, fornecendo instruções claras e o passo a passo para cada receita que ele gerar.
- ⇨ Deverá adaptar cada receita ou plano precisamente às preferências alimentares do utilizador, restrições, equipamento disponível e orçamento.

5.1.1 Resolução A

Para a criação deste modelo "*Culinary-assistant*", apenas precisamos de seguir os passos que foram demonstrados no tutorial, no exemplo da criação do modelo "*SISTCA-Team4-Nutrition*" e fazer os ajustes de forma a seguirmos o que é pedido no enunciado.

- ⇨ **Se realizou o tutorial antes de resolver este exercício:**

Se foi acompanhando e realizando o tutorial do nosso guião, então já terá criado a sua conta no Website do *Ollama* o que significa que para concluir este exercício apenas precisa de concluir as restantes etapas:

1) Criar e guardar um ficheiro *Modelfile* (com o nome ligeiramente diferente do original, por exemplo, Modelfile_1, uma vez que são ficheiros diferentes) no qual terá:

    a. No parâmetro FROM -> *"qwen3:0.6b"* uma vez que no enunciado do exercício é indicado que este é o modelo base que deve ser utilizado.

    b. No parâmetro SYSTEM -> um *system prompt* que indique ao modelo as suas instruções, nomeadamente, as que são indicadas no enunciado. O conteúdo do *system prompt* pode variar um dependendo de quem o contruiu, no entanto, um bom exemplo de um *system prompt* seria algo parecido com:

> *"You are the "Culinary-assistant", a culinary assistant specialized in generating everyday recipes. Your primary function is to help users create practical recipes tailored to their dietary preferences, restrictions, available equipment, and budget. Your responses must be clear and accurate, providing easy-to-follow step-by-step instructions for each recipe you generate. Always adapt recipes precisely according to the user's preferences, restrictions, equipment availability, and budget. Maintain a friendly and encouraging tone, motivating users to explore new recipes and cooking techniques. Whenever you receive a request, ask for the available ingredients, dietary preferences, restrictions, equipment, and budget if the user has not already provided this information."*

2) Executar o comando "`ollama create`" para criar o modelo com base no *Modelfile* anterior. O comando ficaria da seguinte forma:

```
ollama create <Username>/Culinary-assistant -f Modelfile_1
```

Após a realização destas 2 etapas, o modelo será criado com sucesso e aparecerá no *Open WebUI* (Figura 25), concluindo assim o primeiro exercício.

⇨ **Se <u>não</u> realizou o tutorial antes de resolver este exercício:**

Caso não tenha realizado o tutorial, então será necessário realizar algumas etapas e só depois é que irá conseguir criar o do modelo "*Culinary-assistant*".

As etapas de resolução deste exercício são as seguintes:

1) Criar conta *website oficial do Ollama*;

2) Criar um diretório com exatamente o mesmo nome de utilizador da conta do *Ollama* que foi anteriormente criada;

3) Criar e guardar um ficheiro *Modelfile* (com o nome ligeiramente diferente do original, por exemplo, Modelfile_1, uma vez que são ficheiros diferentes) no qual terá:

   a. No parâmetro FROM -> *"qwen3:0.6b"* uma vez que no enunciado do exercício é indicado que este é o modelo base que deve ser utilizado.

   b. No parâmetro SYSTEM -> um *system prompt* que indique ao modelo as suas instruções, nomeadamente, as que são indicadas no enunciado. O conteúdo do *system prompt* pode variar um dependendo de quem o contruiu, no entanto, um bom exemplo de um *system prompt* seria algo parecido com:

> *"You are the "Culinary-assistant", a culinary assistant specialized in generating everyday recipes. Your primary function is to help users create practical recipes tailored to their dietary preferences, restrictions, available equipment, and budget. Your responses must be clear and accurate, providing easy-to-follow step-by-step instructions for each recipe you generate. Always adapt recipes precisely according to the user's preferences, restrictions, equipment availability, and budget. Maintain a friendly and encouraging tone, motivating users to explore new recipes and cooking techniques. Whenever you receive a request, ask for the available ingredients, dietary preferences, restrictions, equipment, and budget if the user has not already provided this information."*

4) Executar o comando "`ollama create`" para criar o modelo com base no *Modelfile* anterior. O comando ficaria da seguinte forma:

```
ollama create <Username>/Culinary-assistant -f Modelfile_1
```

Após a realização destas 4 etapas, o modelo será criado com sucesso e aparecerá no *Open WebUI* (Figura 25), concluindo assim o primeiro exercício.

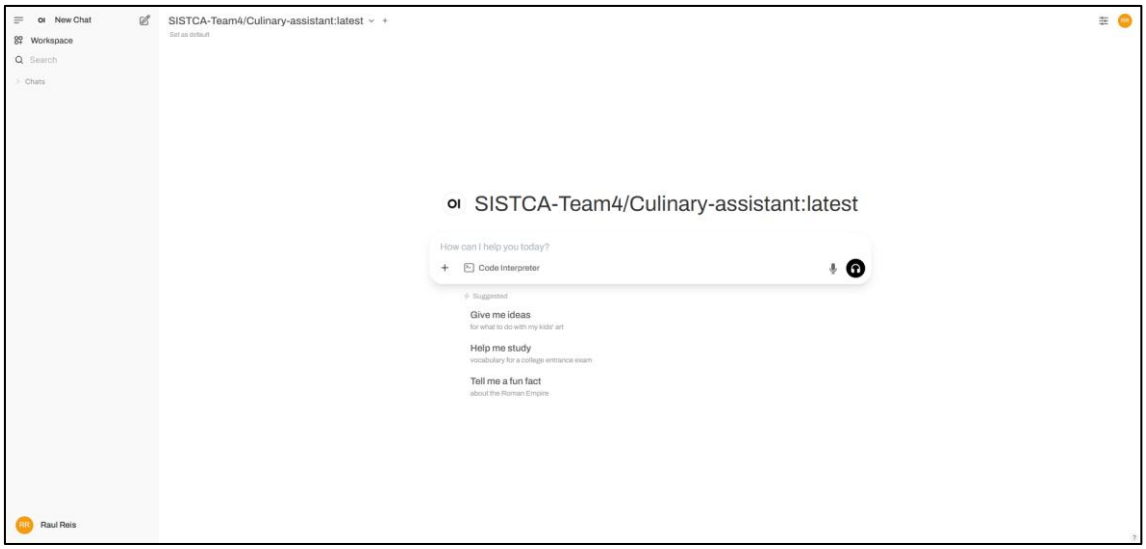


Figura 25 - Culinary-assistant LLM no Open WebUI

5.2 Problema B

É proposto para o segundo exercício a publicação deste modelo, "*Culinary-assistant*", no website do *Ollama* de forma que ele se torne público para quem tiver interesse em fazer *download*.

5.2.1 Resolução B

Tal como no exercício anterior, para fazer a publicação deste modelo "*Culinary-assistant*", devemos seguir os passos que foram demonstrados no tutorial, no exemplo da publicação do modelo "*SISTCA-Team4-Nutrition*" e fazer os ajustes necessários.

⇨ **Se realizou o tutorial antes de resolver este exercício:**

Se foi acompanhando e realizando o tutorial do nosso guião, então já terá criado a sua *Public Key*, o que significa que para concluir este exercício apenas precisa de executar o comando "`push`" para publicar o novo modelo "*Culinary-assistant*" no website do *Ollama*:

```
ollama push <Username>/Culinary-assistant:latest
```

Após a execução deste comando, uma vez que já tinha a *Public Key* gerada e inserida no website do *Ollama*, o modelo será publicado com sucesso e aparecerá no *Website* (Figura 26), concluindo assim o segundo exercício.

⇨ **Se <u>não</u> realizou o tutorial antes de resolver este exercício:**

Caso não tenha realizado o tutorial, então será necessário realizar algumas etapas e só depois é que irá conseguir fazer a publicação do modelo "*Culinary-assistant*" no website do *Ollama*.

As etapas de resolução deste exercício são as seguintes:

1) Gerar uma *Public Key* no terminal com o seguinte comando:

```
ssh-keygen -t ed25519 -N ""
```

2) Exibir a *Public Key* gerada com o comando:

```
sudo cp ~/.ssh/id_ed25519 /usr/share/ollama/.ollama
cat ~/.ssh/id_ed25519.pub
```

3) Copiar e colar a *Public Key* na secção de "*Ollama keys*" no website do *Ollama*;

4) Fazer *push* ao modelo "*Culinary-assistant*" no nosso *namespace*:

```
ollama push <Username>/Culinary-assistant:latest
```

Após a realização destas 4 etapas, o modelo será publicado com sucesso e aparecerá no *Website* do *Ollama* (Figura 26), concluindo assim o segundo exercício.

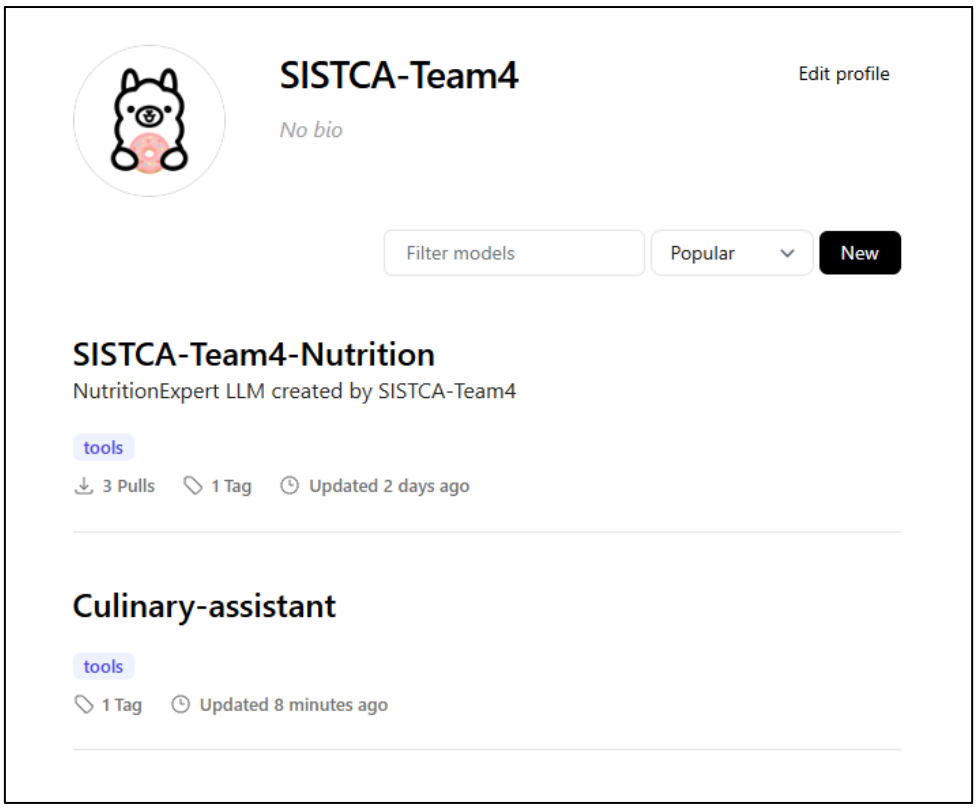


Figura 26 - Culinary-assistant LLM Publicado no Website do Ollama

6. Desafio

No âmbito deste desafio final, pretende-se consolidar os conhecimentos adquiridos ao longo dos exercícios anteriores, aplicando-os num novo contexto temático. O objetivo consiste na criação de um LLM com funções de *Personal-Trainer*, cuja finalidade é auxiliar na elaboração de planos de treino personalizados, ajustados às características e metas dos utilizadores. Para tal, deverá ser selecionado um modelo base compatível com os recursos computacionais disponíveis, bem como elaborado um *system prompt* claro e estruturado, que oriente o modelo a adotar uma postura especializada na área do treino físico. O modelo deve receber instruções que lhe indicam que:

- ⇨ Deve especializar-se em fornecer planos de treinamento físico personalizados com base nas metas específicas do utilizador, como perda de gordura, ganho de massa muscular ou condicionamento geral.

- ⇨ As suas respostas devem incluir instruções claras e práticas para cada exercício ou rotina de treino recomendada, garantindo que sejam compreensíveis e fáceis de seguir.

- ⇨ Cada plano de treino deve ser adaptado às necessidades individuais do utilizador, levando em consideração fatores como condição física atual, preferências de treino, disponibilidade de equipamento e quaisquer limitações físicas.

Adicionalmente, este desafio propõe a criação de uma estratégia de interação eficaz com o modelo, de forma a maximizar a sua utilidade enquanto assistente virtual na área do treino físico. Sugere-se, para isso, a **criação de um ou mais *Prompts***, com o intuito de evitar copiar e colar várias vezes mensagens repetidas durante uma conversa com o LLM. Para esclarecer melhor, vamos supor o seguinte exemplo:

O João quer utilizar o seu modelo *Personal-Trainer* para criar um plano de treino personalizado para cada elemento da sua família. A mensagem que ele vai entregar ao LLM para que ele criar um plano de treino personalizado é a seguinte:

> *"Create a complete workout plan for [goal: fat loss, muscle gain, toning], considering that the user has [available equipment: dumbbells, barbell, bodyweight] as available equipment, and wants to train [weekly frequency: 3x, 4x, 5x] times per week."*

Assumindo que na sua família existem 6 pessoas, o João terá de copiar e colar esta mensagem 6 vezes e editá-la para cada elemento da sua família, assumindo que todos têm objetivos, e disponibilidades para treinar, diferentes.

Se o João tivesse previamente criado um ***Prompt*** com a sua mensagem lá, ele não teria a necessidade de copiar e colar a mensagem 6 vezes para cada elemento da família, uma vez que poderia simplesmente evocar na sua conversa com o LLM esse mesmo ***Prompt*** que iria colar a mensagem no chat por ele, sem ele precisar de se preocupar em ir buscá-la. A ideia com a criação de um ***Prompt*** é tornar processos semelhantes a este mais dinâmicos e automatizados. No Open WebUI existe uma funcionalidade que permite criar este tipo de ***Prompt*** com este objetivo em mente.

Dica: Para encontrar esta funcionalidade comece por explorar a secção "*Workspace*" no *Open WebUI*.

Extra: Queres testar o quanto aprendeste com este guião? Realiza o *Kahoot* que preparamos para ti e testa o teu conhecimento sobre *LLMs*, Ollama e Open WebUI!

Kahoot!

LLMs Kahoot Quiz - SISTCA

7. Referências

[1] Ollama, "Ollama ReadMe," [Online]. Available: https://github.com/ollama/ollama/blob/main/docs/README.md.

[2] P. Moses, "BELSTERNS Technologies," [Online]. Available: https://www.belsterns.com/post/ollama-vs-llama-cpp-which-one-should-you-choose-in-2025.

[3] L. Studio, "LM Studio Docs," [Online]. Available: https://lmstudio.ai/docs/app.

[4] L. Studio, "lmstudio-ai - lms github," LM Studio, [Online]. Available: https://github.com/lmstudio-ai/lms. [Acedido em 6 4 2025].

[5] E. M. H. Monsur, "Medium," [Online]. Available: https://medium.com/ml-and-dl/ollama-vs-lm-studio-which-ai-platform-is-best-for-your-project-c58e384fa2e4.

[6] vLLM, "vLLM Documentation," [Online]. Available: https://docs.vllm.ai/en/latest/.

[7] Koboldcpp, "Doc Oficial Koboldcpp," [Online]. Available: https://github.com/awtrisk/koboldcpp.

[8] E. Hellstrom, "LM Studio vs. Ollama: Choosing the Right Local LLM Platform," 21 3 2025. [Online]. Available: https://blog.promptlayer.com/lm-studio-vs-ollama-choosing-the-right-local-llm-platform/. [Acedido em 2025 4 6].

[9] H. Navarro, "Ollama vs vLLM: which framework is better for inference? 👊 (Part II)," 3 2 2025. [Online]. Available: https://henrynavarro.org/ollama-vs-vllm-which-framework-is-better-for-inference-part-ii-37f7e24d3899. [Acedido em 2025 4 6].

[10] O. WebUI, "Open WebUI Docs," [Online]. Available: https://docs.openwebui.com/. [Acedido em 2025 4 6].

[11] G. Rodewald, "Medium," 24 Março 2025. [Online]. Available: https://medium.com/@gabrielrodewald/running-models-with-ollama-step-by-step-60b6f6125807.

8. Anexos

Modelfile.txt Modelfile_1.txt